

ERP System — Complete Design Specification

Multi-Module Sales / Inventory / Dispatch / Finance ERP

Prepared for: Build kickoff (pre-code design phase) **Stack:** React 18 + Vite + Tailwind + Zustand + TanStack Query + Socket.io Client | Node.js + Express + Sequelize + MySQL + JWT + Socket.io

0. Assumptions & Open Items (Please Confirm Before Code Phase)

No source specification document was attached to this conversation, so the 17-module breakdown and the Stock1/Stock2 semantics below are **inferred** from the roles, alerts, and workflows you listed (credit limits, reorder, smart suggestions, dispatch, Excel import, PDF generation). Please correct anything that doesn't match your actual spec — everything downstream (DB schema, APIs, code) will be generated from this document, so this is the cheapest point to fix it.

Inferred 17 Modules:

1. Authentication & User Management
2. Role & Permission Management
3. Dashboard (per-role)
4. Customer Management
5. Vendor / Supplier Management
6. Product & Catalog Management
7. Inventory & Stock Management (Stock1 / Stock2)
8. Sales & Order Management
9. Dispatch Management
10. Payment & Finance Management
11. Credit Limit Management

- 12. Reorder Management
- 13. Smart Suggestion Engine
- 14. Notification System
- 15. Audit Log System
- 16. Excel Import / Export
- 17. PDF Generation & Reporting

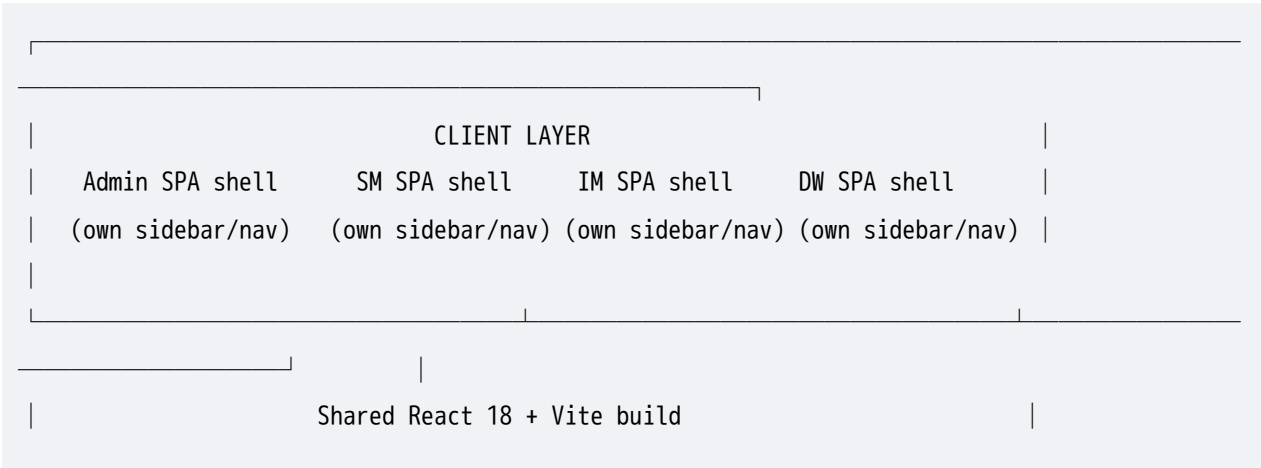
Stock1 / Stock2 — assumed meaning:

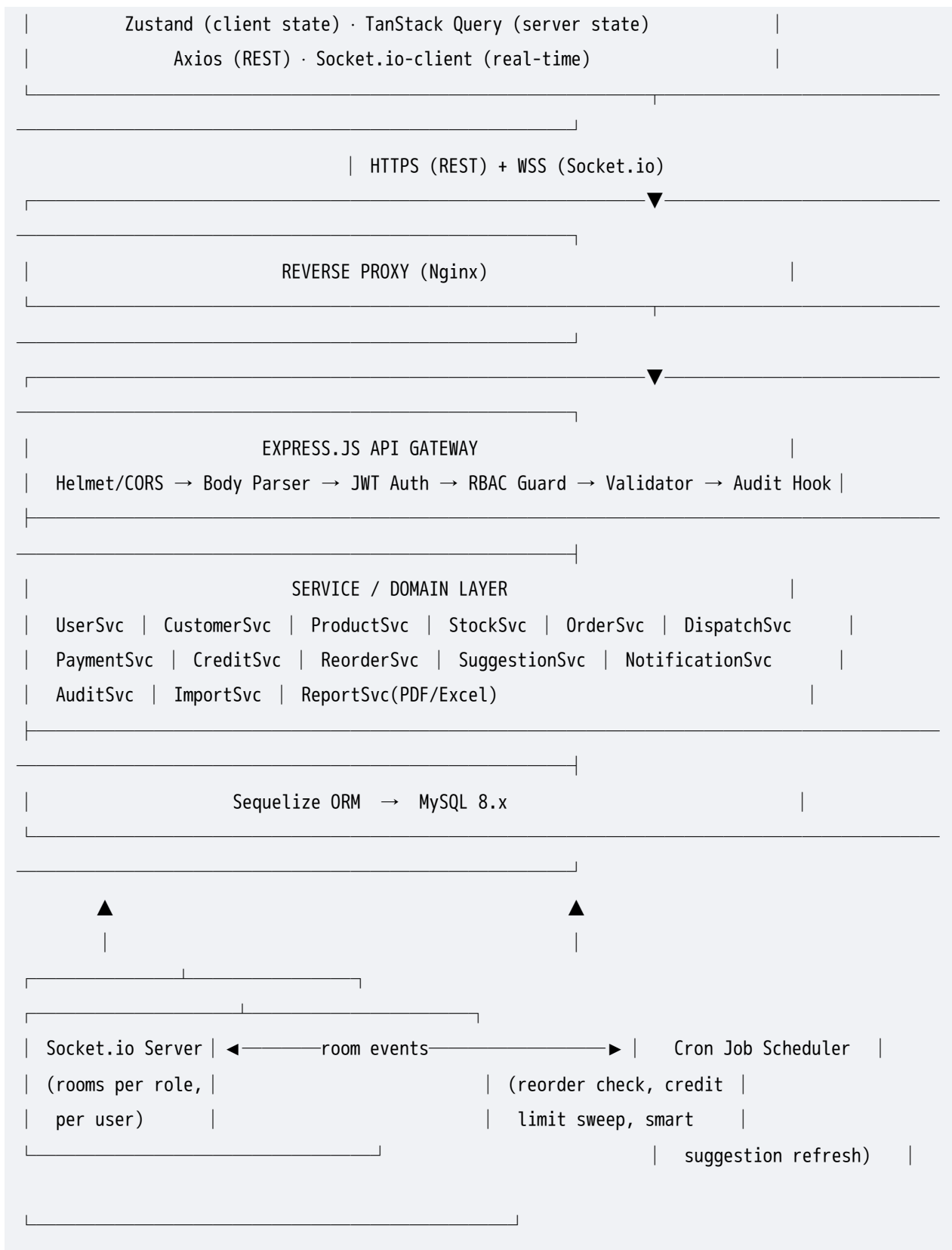
- **Stock1 = Warehouse/Godown Stock** — the real, physical, owned inventory. Increases on purchase/GRN or returns. Owned and edited by the Inventory Manager.
- **Stock2 = Allocated/Reserved Stock** — quantity earmarked the moment a Sales Manager's order is confirmed, moved out of "available" Stock1 but not yet physically dispatched. Decrement to zero (final stock-out) only when Dispatch completes.
- "Available to sell" = Stock1 – Stock2(reserved against open orders).

If in your spec Stock1/Stock2 instead means something like *two separate physical warehouses, or raw vs. finished goods*, the schema in Section 5 and the workflow in Section 13 will need to change — the rest of the architecture (roles, APIs, sockets, notifications) stays valid either way.

1. Complete System Architecture

The system is a **single codebase, multi-tenant-by-role** architecture: one Express backend, one MySQL database, one React build — but every role gets a fully distinct experience via role-scoped routing, layout shells, and permission-gated APIs. Nothing is technically four apps; everything is *designed* to feel like four apps.





Key architectural principles:

- **Single source of truth for permissions:** every UI affordance (sidebar item, button, route) is driven by a permission token returned at login — never hardcoded per component.

- **Stateless API, stateful sockets:** REST calls carry JWT each time; Socket.io connections are authenticated once at handshake and joined into rooms.
 - **Service layer owns business rules:** controllers stay thin (parse request → call service → format response); all cross-cutting logic (stock reservation, credit checks, audit writes) lives in services so it's reusable from REST, sockets, and cron jobs alike.
 - **Everything mutating state goes through the audit hook** — this is a cross-cutting middleware/service wrapper, not a per-route afterthought.
-

2. Module Dependency Diagram

```
graph TD
  Auth[Auth & User Mgmt] --> RBAC[Role & Permission Mgmt]
  RBAC --> Dashboard[Dashboard]
  RBAC --> Customer[Customer Mgmt]
  RBAC --> Vendor[Vendor Mgmt]
  RBAC --> Product[Product/Catalog Mgmt]
  Product --> Inventory[Inventory Stock1/Stock2]
  Customer --> Credit[Credit Limit Mgmt]
  Customer --> Orders[Sales & Order Mgmt]
  Product --> Orders
  Inventory --> Orders
  Credit --> Orders
  Orders --> Dispatch[Dispatch Mgmt]
  Inventory --> Dispatch
  Orders --> Payment[Payment & Finance]
  Credit --> Payment
  Inventory --> Reorder[Reorder Mgmt]
  Vendor --> Reorder
  Orders --> Smart[Smart Suggestion Engine]
  Inventory --> Smart
  Inventory --> Notif[Notification System]
  Credit --> Notif
  Orders --> Notif
  Dispatch --> Notif
  Reorder --> Notif
  Auth --> Audit[Audit Log System]
  Orders --> Audit
  Inventory --> Audit
```

```

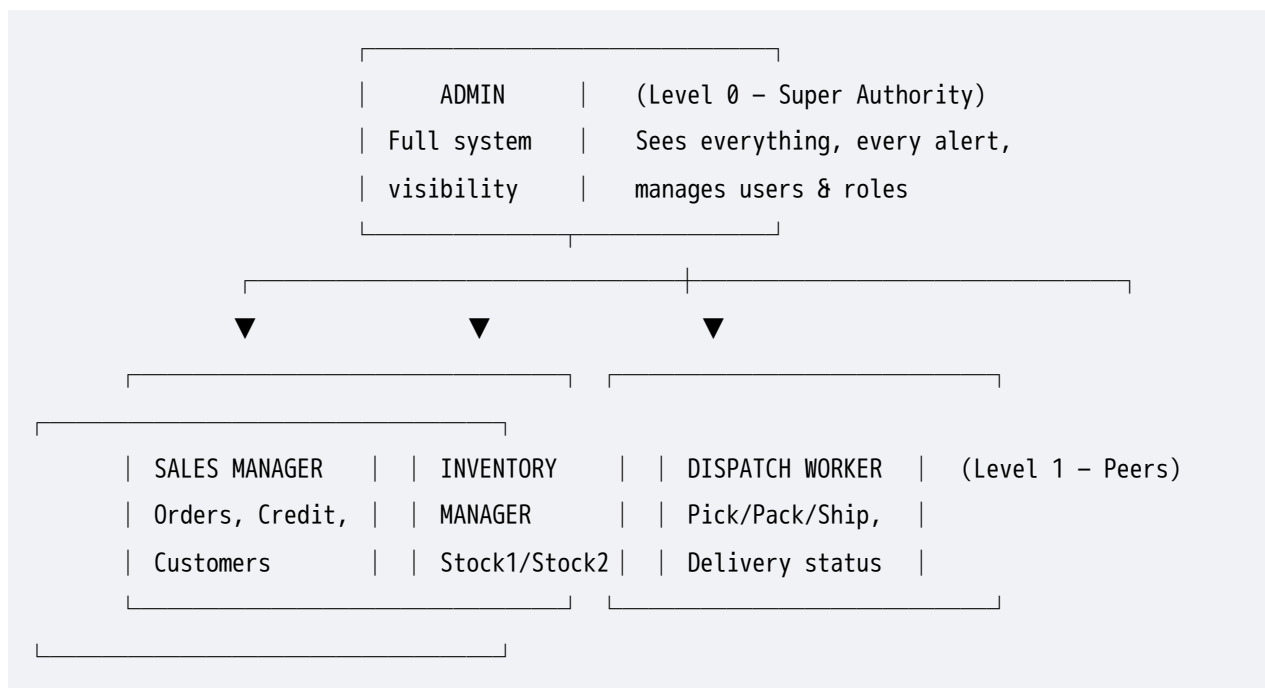
Payment --> Audit
Product --> Excel[Excel Import/Export]
Customer --> Excel
Inventory --> Excel
Orders --> PDF[PDF Generation & Reporting]
Dispatch --> PDF
Payment --> PDF
Audit --> PDF

```

Dependency rules this enforces:

- Nothing can depend "backward" into Auth/RBAC — they're foundational and load first.
- Orders is the busiest module: it cannot be built until Customer, Product, Inventory, and Credit exist (in terms of data model, not necessarily UI).
- Dispatch and Payment are downstream of Orders — they read order state, they don't create it.
- Notification, Audit, Excel, and PDF are **cross-cutting** modules — every other module calls into them, they call into nothing else.

3. User Role Hierarchy



- **Admin is the only Level-0 role.** No role reports to another peer role — SM, IM, and DW are *functionally* siloed and cannot see or act on each other's modules except where a workflow explicitly hands off (e.g., a confirmed Order becomes visible to IM/DW only in the dispatch queue, never the full Sales module).
- Admin inherits **read access everywhere** and **write access to Users, Roles, and system Settings only** — Admin generally *oversees* rather than *performs* day-to-day SM/IM/DW transactional work, but is never blocked from intervening (e.g., force-cancel an order, override a credit hold).
- There is no Level-2; this is intentionally a flat operational hierarchy under one administrative layer.

4. Role Permission Matrix

Legend: **F** = Full (Create/Read/Update/Delete) · **C** = Create+Read+Update (no delete) · **R** = Read-only · **O** = Own-records-only · **A** = Approve/Action · **—** = No access

Module	Admin	Sales Manager	Inventory Manager	Dispatch Worker
User Management	F	—	—	—
Roles & Permissions	F	—	—	—
Dashboard (own)	F	R (own widgets)	R (own widgets)	R (own widgets)
Customer Management	F	O (create/edit own customers)	R	—
Vendor Management	F	—	C	—
Product & Catalog	F	R	F	R
Inventory — Stock1	R	—	F	R

Inventory — Stock2	R	R (own orders' reservations)	F	C (consume on dispatch)
Sales & Order Mgmt	F (override)	O (own orders, full lifecycle)	R	R (assigned orders only)
Credit Limit Mgmt	F	R + request increase	—	—
Dispatch Management	F (override)	R (status only)	R	O (own assigned dispatches)
Payment & Finance	F	C (record against own orders)	—	—
Reorder Management	F	—	F	—
Smart Suggestion Engine	R (all)	R (sales-relevant)	R (stock-relevant)	—
Notifications	R (all categories)	R (own scope)	R (own scope)	R (own scope)
Audit Logs	F (view + export)	—	—	—
Excel Import/Export	F	— (export customers only)	F (products/stock import)	—
PDF Generation/Reports	F (all reports)	C (invoice, own orders)	C (stock reports)	C (dispatch note)

This matrix maps 1:1 to a `role_permissions` table (Section 5) and is enforced server-side in middleware — the frontend matrix (Section 22/23) is a *reflection* of this table, not an independent source of truth.

5. Complete Database Design

Naming convention: snake_case tables, singular logical entities, `id` PK (BIGINT UNSIGNED AUTO_INCREMENT), `created_at` / `updated_at` on every table, soft-delete (`deleted_at`) on master-data tables.

5.1 Identity & Access

users `id`, `name`, `email` (unique), `password_hash`, `role_id` (FK), `phone`, `is_active`, `last_login_at`, `created_at`, `updated_at`

roles `id`, `name` (admin|sales_manager|inventory_manager|dispatch_worker), `description`

permissions `id`, `module`, `action` (create|read|update|delete|approve), `code` (unique e.g. orders.create)

role_permissions `id`, `role_id` (FK), `permission_id` (FK) — implements the matrix in Section 4 as data, not code.

refresh_tokens `id`, `user_id` (FK), `token_hash`, `expires_at`, `revoked_at`, `ip_address`

5.2 Customer / Vendor / Product

customers `id`, `name`, `code` (unique), `phone`, `email`, `billing_address`, `shipping_address`, `gst_number`, `credit_limit_amount`, `credit_used_amount`, `payment_terms_days`, `assigned_sales_manager_id` (FK users), `is_active`, `deleted_at`

vendors `id`, `name`, `code` (unique), `phone`, `email`, `address`, `gst_number`, `is_active`

product_categories `id`, `name`, `parent_category_id` (self-FK, nullable)

products `id`, `sku` (unique), `name`, `category_id` (FK), `unit` (pcs|box|kg...), `unit_price`, `cost_price`, `reorder_threshold`, `reorder_quantity`, `preferred_vendor_id` (FK vendors), `is_active`, `deleted_at`

5.3 Inventory (Stock1 / Stock2)

stock1 (*warehouse/owned stock — one row per product*) id, product_id (FK, unique), quantity_on_hand, last_updated_at

stock2 (*allocated/reserved stock — one row per product*) id, product_id (FK, unique), quantity_reserved, last_updated_at

stock_transactions (*immutable ledger — the real source of truth; stock1/stock2 are derived/cached totals*) id, product_id (FK), transaction_type (PURCHASE_IN|TRANSFER_S1_TO_S2|RELEASE_S2_TO_S1|DISPATCH_OUT|RETURN_IN|ADJUSTMENT), quantity, reference_type (order|dispatch|grn|manual), reference_id, performed_by (FK users), notes, created_at

5.4 Sales & Orders

orders id, order_number (unique), customer_id (FK), sales_manager_id (FK users), status (DRAFT|CONFIRMED|ALLOCATED|DISPATCHED|DELIVERED|CLOSED|CANCELLED|ON_HOLD), order_date, total_amount, credit_hold (bool), created_at

order_items id, order_id (FK), product_id (FK), quantity, unit_price, line_total

order_status_history id, order_id (FK), from_status, to_status, changed_by (FK users), reason, created_at

5.5 Dispatch

dispatches id, dispatch_number (unique), order_id (FK), dispatch_worker_id (FK users), status (QUEUED|PICKING|PACKED|DISPATCHED|DELIVERED), dispatched_at, delivered_at

dispatch_items id, dispatch_id (FK), order_item_id (FK), product_id (FK), quantity_dispatched

5.6 Payments & Credit

payments id, payment_number (unique), order_id (FK, nullable for advance payments), customer_id (FK), amount, mode (cash|bank|cheque|upi), reference_no, received_by (FK users), payment_date

credit_limit_history id, customer_id (FK), old_limit, new_limit, changed_by (FK users), reason, created_at

5.7 Reorder & Smart Suggestions

reorder_suggestions id, product_id (FK), current_stock1_qty, threshold, suggested_quantity, status (NEW|ACKNOWLEDGED|CONVERTED|DISMISSED), created_at

smart_suggestions id, target_role (sales_manager|inventory_manager), suggestion_type (REORDER_PRODUCT|UPSELL_CUSTOMER|SLOW_MOVING_STOCK|REPEAT_ORDER_DUE), entity_type, entity_id, payload (JSON), status (NEW|ACTED|DISMISSED), created_at

5.8 Notifications & Audit

notifications id, type, title, message, severity (info|warning|critical), payload (JSON), created_by, created_at

notification_recipients id, notification_id (FK), user_id (FK, nullable), role_id (FK, nullable – for broadcast-to-role), is_read, read_at

audit_logs id, actor_id (FK users), actor_role, action_type (create|update|delete|approve|login|export), module, entity_type, entity_id, before_state (JSON), after_state (JSON), ip_address, created_at

5.9 Import / Export / Documents

excel_import_logs id, import_type (products|customers|opening_stock), file_name, uploaded_by (FK users), total_rows, success_rows, failed_rows, error_report_path, status (PROCESSING|COMPLETED|FAILED), created_at

generated_documents id, doc_type (invoice|dispatch_note|receipt|stock_report|audit_report), reference_type, reference_id, file_path, generated_by (FK users), created_at

Indexing notes: composite index on (status, sales_manager_id) for orders; (status, dispatch_worker_id) for dispatches; (product_id) unique on stock1/stock2; (customer_id, created_at) on payments; (module, created_at) on audit_logs for fast filtered audit queries.